注册人脸识别

注册人脸识别
- 例程实验效果
- 代码讲解
  - 人脸注册部分
  - 人脸识别部分
- 拓展
  - 创建目录部分

例程实验效果

最后我们来学习一下用K230实现人脸识别的功能。

人脸识别分为【注册】和【识别】两部分，我们需要先对人脸进行注册

例程代码中提供的注册代码，作用是注册某个目录下的所有图片文件，所以我们需要先对想要识别的人脸进行拍照

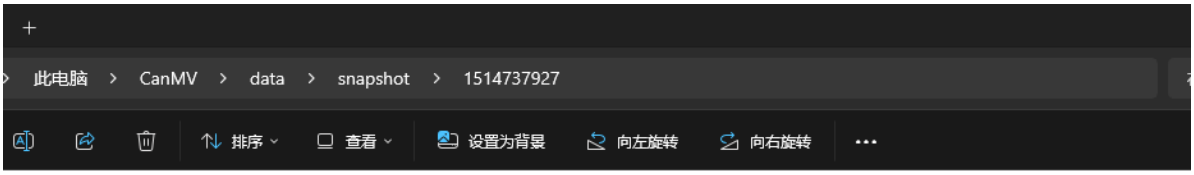
这里我们以基础例程中提供的【照相机】为例，对不同人脸进行拍照

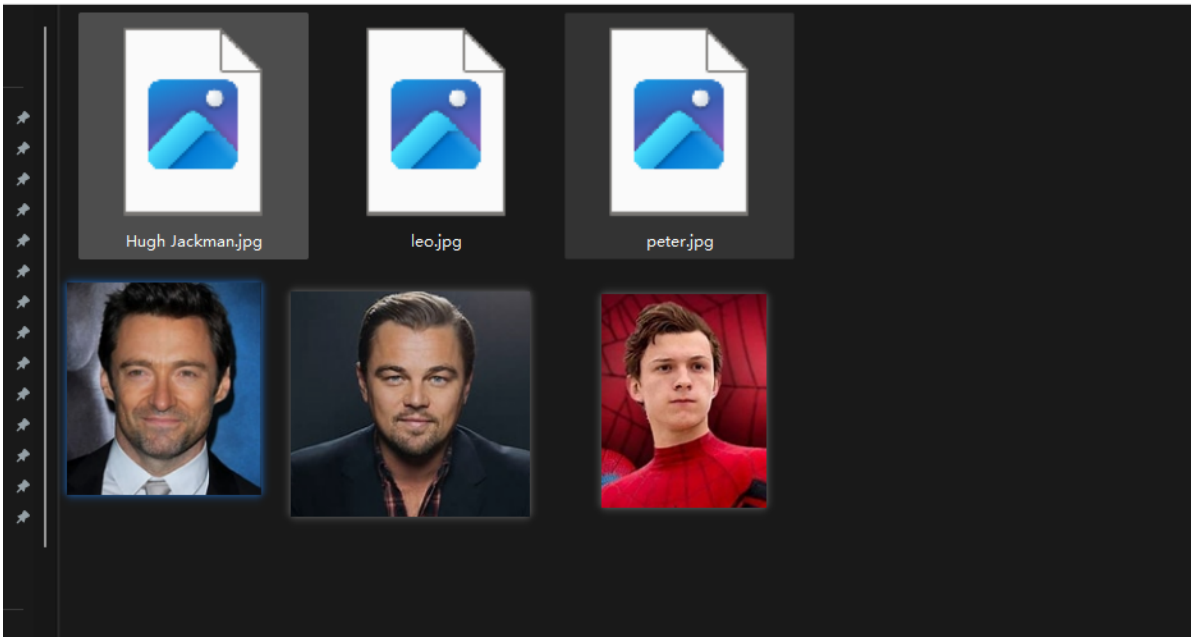






然后我们找到刚刚拍摄的图片保存的地址，将图片文件名称改为人物名称





下一步，我们打开人脸注册的例程代码

修改exce_demo()方法中的database_img_dir变量值，修改为我们拍照保存的路径

在本例程中，路径为【/data/snapshot/1514737927/】

修改好后我们点击左下角运行按钮，等待注册人脸

```
Start ...
已创建目录: /data/face_databse/1514737927/
/data/snapshot/1514737927/leo.jpg
Success!
/data/snapshot/1514737927/peter.jpg
Success!
/data/snapshot/1514737927/Hugh Jackman.jpg
Success!
人脸注册功能退出
```

等待程序运行结束后，我们就可以运行人脸识别例程了。

在CanMV IDE中打开人脸识别例程，

修改exce_demo()中的database_dir变量为【/data/face_databse/1514737927/】

其中【1514737927】是注册人脸时，人脸照片所在的目录名称

点击左下角按钮运行

我们以Peter为例，图片中注册过的Peter会被绿框标记出来，而未注册的人脸则是以蓝框标记



为什么不直接用图片，而是要用K230拍照？因为原图和经K230的摄像头读取后的图片会因为各种环境因素导致存在偏差，用K230拍照后的结果作为训练集，得到的模型会更加适合当前的K230模块来做识别

> 在自行训练模型时也是同理，尽量使用K230拍照作为训练集

> 已添加串口输出功能
>
> 检测到人脸后会发送如下格式的串口输出
>
> 如果是未知的人脸，则发送：
>
> $x,y,w,h,unknown#
>
> 如果是数据库中已注册过的人脸，则发送
>
> $x,y,w,h,name,score#
>
> 其中'$'代表数据的开头，'#'代表数据的结尾
>
> x，y，w，h是人脸检测框的位置（分辨率为640*480）
>
> unknown表示未知的人脸
>
> name表示识别到的人脸的姓名
>
> score表示识别得分，得分越高说明识别正确的可能性越高

代码讲解

由于本章代码过多，不在此处放完整代码源文件，仅留下部分关键代码讲解

人脸注册部分

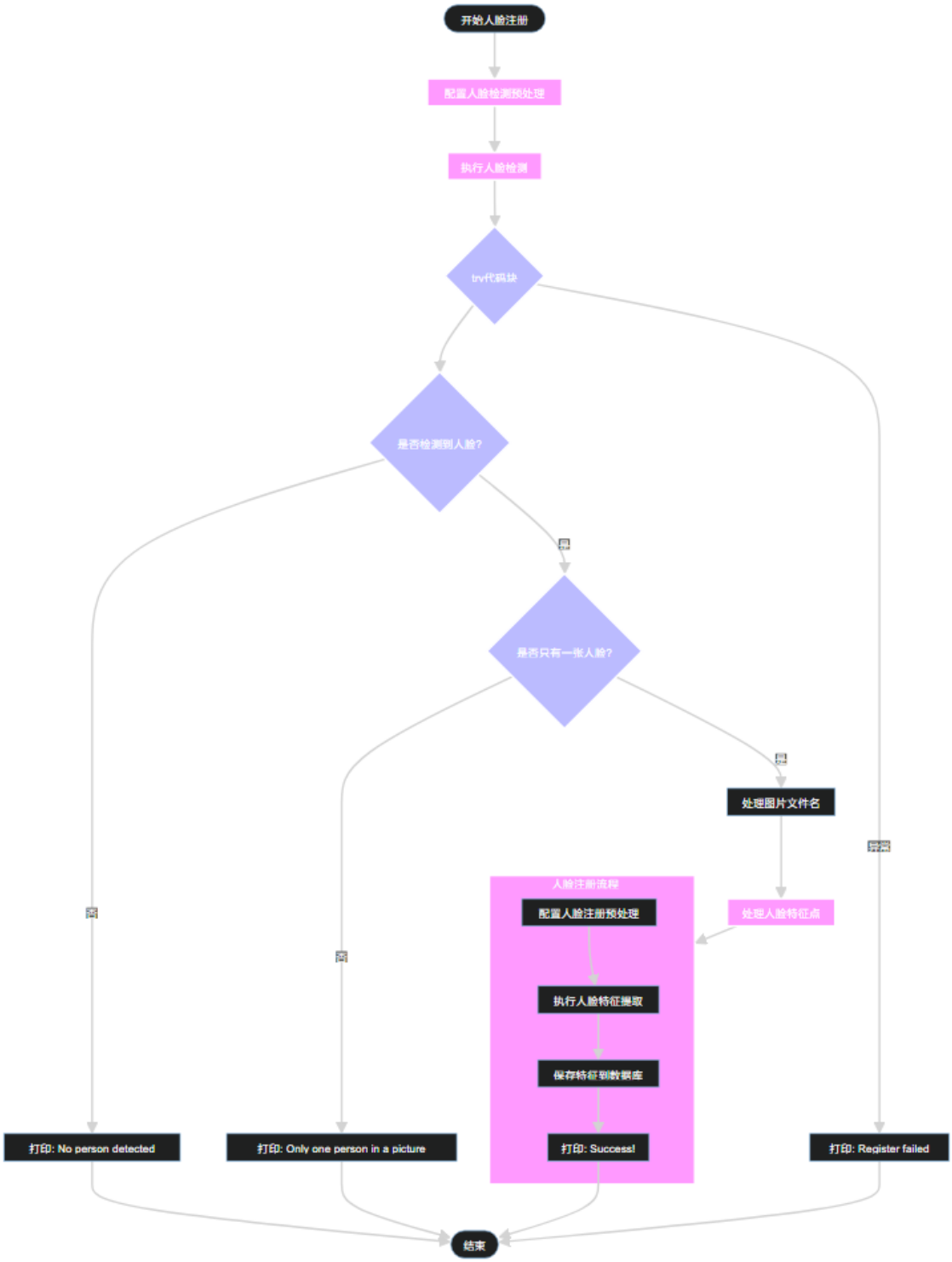
FaceRegistration的run方法

```
    def run(self, input_np, img_file):
        """运行人脸注册流程 / Run face registration process"""
        # 配置人脸检测预处理 / Configure face detection preprocessing
        self.face_det.config_preprocess(input_image_size=
[input_np.shape[3],input_np.shape[2]])
        # 执行人脸检测 / Perform face detection
        det_boxes, landms = self.face_det.run(input_np)
```

```
        try:
            if det_boxes:
                if det_boxes.shape[0] == 1:
                    # 若只检测到一张人脸，进行注册 / If only one face is detected,
proceed with registration
                    db_i_name = img_file.split('.')[0]
                    for landm in landms:
                        # 配置人脸注册预处理 / Configure face registration
preprocessing
                        self.face_reg.config_preprocess(landm, input_image_size=
[input_np.shape[3],input_np.shape[2]])
                        # 执行人脸特征提取 / Perform face feature extraction
                        reg_result = self.face_reg.run(input_np)
                        # 保存特征到数据库 / Save features to database
                        with open(self.database_dir+'{}.bin'.format(db_i_name),
"wb") as file:
                            file.write(reg_result.tobytes())
                            print('Success!')
                else:
                    print('Only one person in a picture when you sign up')
            else:
                print('No person detected')
        except:
            print("Register failed")
```

1. 预处理与检测:
    - 配置人脸检测的预处理参数
    - 执行人脸检测
2. 检测结果处理:
    - 检查是否检测到人脸
    - 确认是否只有一张人脸
3. 注册流程:
    - 处理图片文件名
    - 处理人脸特征点
    - 配置注册预处理
    - 提取人脸特征
    - 保存到数据库
4. 异常处理:
    - 处理注册失败的情况
    - 输出相应的错误信息

完整流程图如下:



人脸识别部分

> 如果报错：
>
> Traceback (most recent call last):
> File "", line 531, in
> File "", line 486, in exce_demo
> File "", line 349, in **init**
> File "", line 374, in database_init
> OSError: [Errno 2] ENOENT
>
> 请检查代码中路径是否填写正确

```
    def run(self, input_np):
        """
        运行人脸识别 / Run face recognition
        """
        # 人脸检测 / Face detection
        det_boxes, landms = self.face_det.run(input_np)
        recg_res = []

        # 对每个检测到的人脸进行识别 / Recognize each detected face
        for landm in landms:
            self.face_reg.config_preprocess(landm)
            feature = self.face_reg.run(input_np)
            res = self.database_search(feature)
            recg_res.append(res)

        return det_boxes, recg_res

    def database_init(self):
        """
        初始化人脸数据库 / Initialize face database
        """
        with ScopedTiming("database_init", self.debug_mode > 1):
            # 读取数据库文件 / Read database files
            db_file_list = os.listdir(self.database_dir)
            for db_file in db_file_list:
                if not db_file.endswith('.bin'):
                    continue
                if self.valid_register_face >= self.max_register_face:
                    break

                valid_index = self.valid_register_face
                full_db_file = self.database_dir + db_file

                # 读取特征数据 / Read feature data
                with open(full_db_file, 'rb') as f:
                    data = f.read()
                feature = np.frombuffer(data, dtype=np.float)
                self.db_data.append(feature)

                # 保存人名 / Save person name
                name = db_file.split('.')[0]
                self.db_name.append(name)
                self.valid_register_face += 1

    def database_reset(self):
        """
        重置数据库 / Reset database
        """
        with ScopedTiming("database_reset", self.debug_mode > 1):
            print("database clearing...")
            self.db_name = []
            self.db_data = []
            self.valid_register_face = 0
            print("database clear Done!")

    def database_search(self, feature):
        """
        在数据库中搜索匹配的人脸 / Search for matching face in database
```

```
        """
        with ScopedTiming("database_search", self.debug_mode > 1):
            v_id = -1
            v_score_max = 0.0

            # 特征归一化 / Feature normalization
            feature /= np.linalg.norm(feature)

            # 遍历数据库进行匹配 / Search through database for matches
            for i in range(self.valid_register_face):
                db_feature = self.db_data[i]
                db_feature /= np.linalg.norm(db_feature)
                v_score = np.dot(feature, db_feature)/2 + 0.5

                if v_score > v_score_max:
                    v_score_max = v_score
                    v_id = i

            # 返回识别结果 / Return recognition result
            if v_id == -1:
                return 'unknown'
            elif v_score_max < self.face_recognition_threshold:
                return 'unknown'
            else:
                result = 'name: {}, score:{}'.format(self.db_name[v_id],
v_score_max)
                return result

    def draw_result(self, pl, dets, recg_results):
        """
        绘制识别结果 / Draw recognition results
        """
        pl.osd_img.clear()
        if dets:
            for i, det in enumerate(dets):
                # 绘制人脸框 / Draw face box
                x1, y1, w, h = map(lambda x: int(round(x, 0)), det[:4])
                x1 = x1 * self.display_size[0]//self.rgb888p_size[0]
                y1 = y1 * self.display_size[1]//self.rgb888p_size[1]
                w = w * self.display_size[0]//self.rgb888p_size[0]
                h = h * self.display_size[1]//self.rgb888p_size[1]

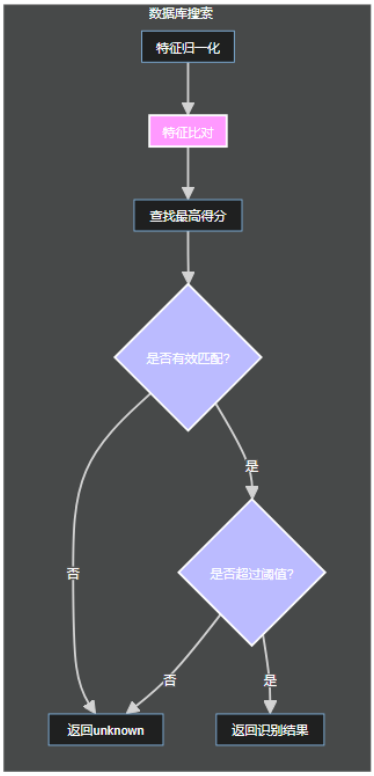
                # 绘制识别结果 / Draw recognition result
                recg_text = recg_results[i]
                if recg_text == 'unknown':
                    pl.osd_img.draw_rectangle(x1, y1, w, h, color=(255,0,0,255),
thickness=4)
                else:
                    pl.osd_img.draw_rectangle(x1, y1, w, h, color=(255,0,255,0),
thickness=4)

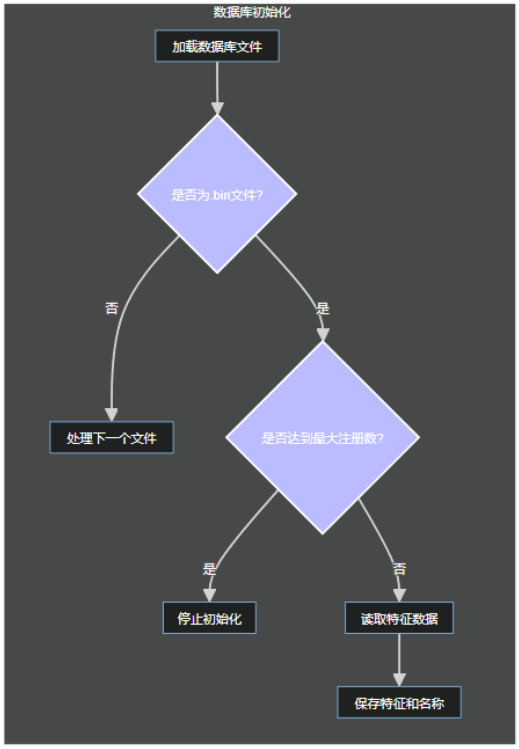
                pl.osd_img.draw_string_advanced(x1, y1, 32, recg_text, color=
(255,255,0,0))
```

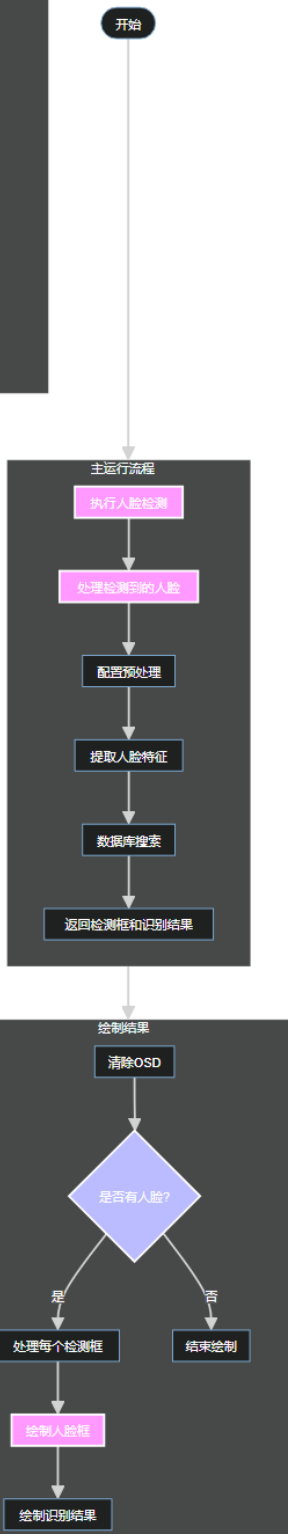
1. 主运行流程(RunFlow):
    - 执行人脸检测
    - 处理检测到的人脸

    - 特征提取和识别
2. 数据库初始化(DBInit):
    - 加载特征文件
    - 检查文件有效性
    - 存储特征数据
3. 数据库搜索(DBSearch):
    - 特征归一化
    - 特征比对
    - 结果判定
4. 结果绘制(DrawResult):
    - 清除旧显示
    - 绘制检测框
    - 显示识别结果

完整流程如图


数据库搜索
特征归一化
特征比对
查找最高得分
是否有效匹配?
否
是
是否超过阈值?
否
是
返回unknown
返回识别结果


数据库初始化
加载数据库文件
是否为.bin文件?
否
是
处理下一个文件
是否达到最大注册数?
是
否
停止初始化
读取特征数据
保存特征和名称


开始
主运行流程
执行人脸检测
处理检测到的人脸
配置预处理
提取人脸特征
数据库搜索
返回检测框和识别结果
绘制结果
清除OSD
是否有人脸?
是
否
处理每个检测框
结束绘制
绘制人脸框
绘制识别结果

拓展

创建目录部分

当你使用K230的os模块去访问一个不存在的目录时，会抛出OSError导致程序结束运行

下面这段代码可以检测指定目录是否存在，若不存则递归的进行创建

```
def ensure_dir(directory):
    """
    递归创建目录
    (Recursively create directory)
    """
    # 如果目录为空字符串或根目录，直接返回
    # (If directory is empty string or root directory, return directly)
    if not directory or directory == '/':
        return

    # 处理路径分隔符，确保使用标准格式
    # (Process path separators to ensure standard format)
    directory = directory.rstrip('/')

    try:
        # 尝试获取目录状态，如果目录存在就直接返回
        # (Try to get directory status, if directory exists then return
directly)
        os.stat(directory)
        print(f'目录已存在: {directory}')
        # (Directory already exists: {directory})
        return
    except OSError:
        # 目录不存在，需要创建
        # (Directory does not exist, need to create)

        # 分割路径以获取父目录
        # (Split path to get parent directory)
        if '/' in directory:
            parent = directory[:directory.rindex('/')]
            if parent and parent != directory:  # 避免无限递归
                                                # (Avoid infinite recursion)
                ensure_dir(parent)

        try:
            # 创建目录
            # (Create directory)
            os.mkdir(directory)
            print(f'已创建目录: {directory}')
            # (Directory created: {directory})
        except OSError as e:
            # 可能是并发创建导致的冲突，再次检查目录是否存在
            # (Possible conflict due to concurrent creation, check again if
directory exists)
            try:
                os.stat(directory)
                print(f'目录已被其他进程创建: {directory}')
                # (Directory has been created by another process: {directory})
            except:
                # 如果仍然不存在，则确实出错了
                # (If it still doesn't exist, there is definitely an error)
```

```
                print(f'创建目录时出错: {e}')
                # (Error creating directory: {e})
    except Exception as e:
        # 捕获其他可能的异常
        # (Catch other possible exceptions)
        print(f'处理目录时出错: {e}')
        # (Error processing directory: {e})
```